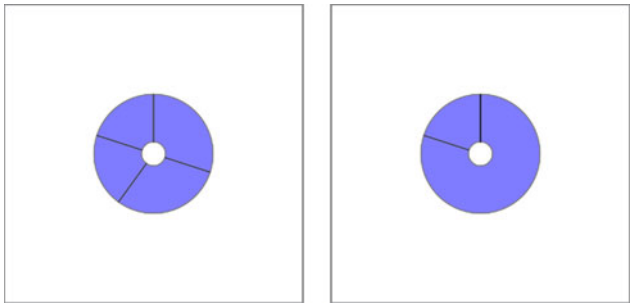


**Figure 5.7 The pie charts representing, on the left, the total number of favorites and, on the right, the total number of retweets**

### 5.2.3 Transitioning

You'll notice that for each value in `nestedTweets`, we totaled the number of tweets, and also used `d3.sum()` to total the number of retweets and favorites (if any). Because we have this data, we can adjust our pie chart to show pie pieces based not on the number of tweets but on those other values. One of the core uses of a layout in D3 is to update the graphical chart. All we need to do is make changes to the data or layout and then rebind the data to the existing graphical elements. By using a transition, we can see the pie chart change from one form to the other. Running the following code first transforms the pie chart to represent the number of favorites instead of the number of tweets. The next block causes the pie chart to represent the number of retweets. The final forms of the pie chart after running that code are shown in figure 5.7.

```
pieChart.value(function(d) {
          return d.numFavorites
});
d3.selectAll("path").data(pieChart(nestedTweets))
         .transition().duration(1000).attr("d", newArc);

pieChart.value(function(d) {return d.numRetweets});
d3.selectAll("path").data(pieChart(nestedTweets))
           .transition().duration(1000).attr("d", newArc);
```

Although the results are what we want, the transition can leave a lot to be desired. Figure 5.8 shows snapshots of the pie chart transitioning from representing the number of tweets to representing the number of favorites. As you'll see by running the code

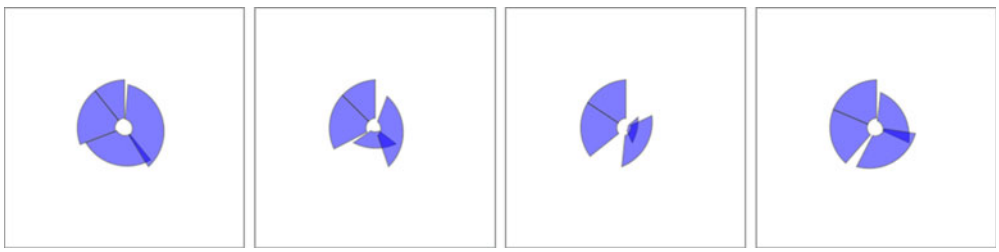


**Figure 5.8 Snapshots of the transition of the pie chart representing the number of tweets to the number of favorites. This transition highlights the need to assign key values for data binding and to use tweens for some types of graphical transition, such as that used for arcs.**